Assignment 2
Learning Theory Write-Up

Dhruv

April 16, 2026

Contents

1 Part A **2**

1.1 Problem A1 . . . 2

1.1.1 1. Restatement . . . 2

1.1.2 2. Explanation . . . 2

1.1.3 3. Solution Plan . . . 2

1.1.4 4. Formal Solution / Proof . . . 2

1.1.5 5. Conclusion . . . 3

1.1.6 6. Lean Formalization . . . 3

1.1.7 7. Computational Check (if applicable) . . . 3

1.2 Problem A2 . . . 3

1.2.1 1. Restatement . . . 3

1.2.2 2. Explanation . . . 4

1.2.3 3. Solution Plan . . . 4

1.2.4 4. Formal Solution / Proof . . . 4

1.2.5 5. Conclusion . . . 5

1.2.6 6. Lean Formalization . . . 5

1.2.7 7. Computational Check (if applicable) . . . 5

1.3 Problem A3 . . . 5

1.3.1 1. Restatement . . . 5

1.3.2 2. Explanation . . . 5

1.3.3 3. Solution Plan . . . 5

1.3.4 4. Formal Solution / Proof . . . 6

1.3.5 5. Conclusion . . . 6

1.3.6 6. Lean Formalization . . . 7

1.3.7 7. Computational Check (if applicable) . . . 7

1.4 Problem A4 . . . 7

1.4.1 1. Restatement . . . 7

1.4.2 2. Explanation . . . 7

1.4.3 3. Solution Plan . . . 7

1.4.4 4. Formal Solution / Proof . . . 7

1.4.5 5. Conclusion . . . 8

1.4.6 6. Lean Formalization . . . 8

1.4.7 7. Computational Check (if applicable) . . . 8

1.5 Problem A5 . . . 8

1.5.1 1. Restatement . . . 8

1.5.2 2. Explanation . . . 8

1.5.3 3. Solution Plan . . . 8

1.5.4 4. Formal Solution / Proof . . . 9
1.5.5 5. Conclusion . . . 9
1.5.6 6. Lean Formalization . . . 9
1.5.7 7. Computational Check (if applicable) . . . 9

2 Part B **9**
2.1 Problem B1 . . . 10
2.1.1 1. Restatement . . . 10
2.1.2 2. Explanation . . . 10
2.1.3 3. Solution Plan . . . 10
2.1.4 4. Formal Solution / Proof . . . 10
2.1.5 5. Conclusion . . . 11
2.1.6 6. Lean Formalization . . . 11
2.1.7 7. Computational Check (if applicable) . . . 11
2.2 Problem B2 . . . 11
2.2.1 1. Restatement . . . 11
2.2.2 2. Explanation . . . 11
2.2.3 3. Solution Plan . . . 12
2.2.4 4. Formal Solution / Proof . . . 12
2.2.5 5. Conclusion . . . 12
2.2.6 6. Lean Formalization . . . 12
2.2.7 7. Computational Check (if applicable) . . . 12
2.3 Problem B3 . . . 13
2.3.1 1. Restatement . . . 13
2.3.2 2. Explanation . . . 13
2.3.3 3. Solution Plan . . . 13
2.3.4 4. Formal Solution / Proof . . . 13
2.3.5 5. Conclusion . . . 14
2.3.6 6. Lean Formalization . . . 14
2.3.7 7. Computational Check (if applicable) . . . 14
2.4 Problem B4 . . . 14
2.4.1 1. Restatement . . . 14
2.4.2 2. Explanation . . . 14
2.4.3 3. Solution Plan . . . 14
2.4.4 4. Formal Solution / Proof . . . 15
2.4.5 5. Conclusion . . . 16
2.4.6 6. Lean Formalization . . . 16
2.4.7 7. Computational Check (if applicable) . . . 16
2.5 Problem B5 . . . 16
2.5.1 1. Restatement . . . 16
2.5.2 2. Explanation . . . 16
2.5.3 3. Solution Plan . . . 16
2.5.4 4. Formal Solution / Proof . . . 17
2.5.5 5. Conclusion . . . 17
2.5.6 6. Lean Formalization . . . 17
2.5.7 7. Computational Check (if applicable) . . . 17

3 Part C **17**

3.1 Problem C1 17
3.1.1 1. Restatement 17
3.1.2 2. Explanation 17
3.1.3 3. Solution Plan 18
3.1.4 4. Formal Solution / Proof 18
3.1.5 5. Conclusion 18
3.1.6 6. Lean Formalization 19
3.1.7 7. Computational Check (if applicable) 19

4 Repository Artifacts and Mapping **19**
4.1 Supporting Repository Artifacts 19
4.2 Write-Up to Repository Mapping 19
4.3 Lean Targets Summary 19

This document gives a complete solution for Parts A, B, and C. It uses the repository style system in `/style/latex/`, includes formal proof plans before each proof, identifies Lean targets for the general linear-representation results, and records a small computational verification for the growth-function counts.

Part A

Throughout Part A, let $\mathcal{H}_{2\text{int}}$ be the class of unions of at most two intervals on the real line. For an ordered sample

$$x_1 < x_2 < \cdots < x_n,$$

we write the induced labeling as a binary string

$$(h(x_1), \ldots, h(x_n)) \in \{0,1\}^n,$$

where $h(x_i) = 1$ means x_i lies in the union of intervals and $h(x_i) = 0$ means it does not.

Problem A1

1. Restatement

For an ordered sample $x_1 < \cdots < x_n$, characterize exactly which binary labelings can be realized by a hypothesis in $\mathcal{H}_{2\text{int}}$. The required answer is that a labeling is realizable if and only if its string contains at most two runs of 1s.

2. Explanation

The question asks for a structural description of the restriction patterns of the class of unions of two intervals. The key concept is that an interval picks out one contiguous block of sample points, so a union of two intervals can pick out at most two such blocks. On a sorted sample, contiguous blocks are exactly runs of 1s in the binary labeling.

3. Solution Plan

1. Prove the forward implication: every hypothesis in $\mathcal{H}_{2\text{int}}$ induces at most two runs of 1s.
2. Prove the reverse implication: from any labeling with at most two runs of 1s, explicitly construct one or two intervals that realize it.
3. Conclude the desired equivalence.

4. Formal Solution / Proof

Theorem 1. *Let $x_1 < \cdots < x_n$ be distinct real numbers. A binary labeling $(y_1, \ldots, y_n) \in \{0,1\}^n$ is realizable by $\mathcal{H}_{2\text{int}}$ if and only if the string $y_1 \ldots y_n$ has at most two runs of 1s.*

Proof. First suppose $h \in \mathcal{H}_{2\text{int}}$. Then there exist intervals I_1 and I_2 such that

$$h = \mathbf{1}_{I_1 \cup I_2}.$$

For a sorted sample, the set of indices i with $x_i \in I_1$ is either empty or a contiguous block of indices, because an interval cannot leave and then re-enter the ordered sample. The same is true

for I_2. Therefore the set of indices with $h(x_i) = 1$ is the union of at most two contiguous blocks. Equivalently, the binary string $h(x_1) \dots h(x_n)$ has at most two runs of 1s.

Now suppose a binary string $y_1 \dots y_n$ has at most two runs of 1s. We construct a hypothesis in $\mathcal{H}_{2\text{int}}$ realizing it.

If there are no runs of 1s, then the all-zero labeling is realized by the empty union of intervals.

If there is exactly one run of 1s, say from indices a through b, then choose any interval containing $x_a, \dots, x_b$ and excluding all other sample points, for example

$$I_1 = \left[\frac{x_{a-1} + x_a}{2}, \frac{x_b + x_{b+1}}{2} \right]$$

with the obvious endpoint modifications when $a = 1$ or $b = n$. Then $\mathbf{1}_{I_1}(x_i) = y_i$ for all i, so the labeling is realized by one interval.

If there are exactly two runs of 1s, say on index blocks $[a, b]$ and $[c, d]$ with $b < c$, then define intervals I_1 and I_2 separately around those two blocks:

$$I_1 = \left[\frac{x_{a-1} + x_a}{2}, \frac{x_b + x_{b+1}}{2} \right], \qquad I_2 = \left[\frac{x_{c-1} + x_c}{2}, \frac{x_d + x_{d+1}}{2} \right],$$

again with endpoint modifications when a block touches the sample boundary. These intervals realize exactly those two runs and no others, so

$$\mathbf{1}_{I_1 \cup I_2}(x_i) = y_i \qquad \text{for all } i.$$

Thus every labeling with at most two runs of 1s is realizable by $\mathcal{H}_{2\text{int}}$, and every realizable labeling has at most two runs of 1s. □

5. Conclusion

The restriction patterns of $\mathcal{H}_{2\text{int}}$ are exactly the binary strings with at most two runs of 1s. This characterization is the foundation for the growth-function count in Problem A2 and the VC-dimension computation in Problem A3.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

For $n \leq 6$, the script `hw2/code/enumeration_check.py` enumerates all binary strings of length n and confirms that the realizable ones are exactly those with at most two runs of 1s.

Problem A2

1. Restatement

Using the characterization from Problem A1, compute the growth function $\Gamma_{\mathcal{H}_{2\text{int}}}(n)$, the number of distinct labelings of n ordered points realizable by $\mathcal{H}_{2\text{int}}$. The count must be broken into the exact cases of no runs, one run, and two runs of 1s.

2. Explanation

The key concept is combinatorial counting. Since Problem A1 gives a complete description of realizable labelings, the growth function is obtained by counting binary strings with at most two runs of 1s and making sure the cases do not overlap.

3. Solution Plan

1. Count the unique string with no 1-run.
2. Count strings with exactly one run of 1s by choosing the start and end of the block.
3. Count strings with exactly two runs of 1s by choosing four boundary positions.
4. Add the three disjoint case counts and simplify.

4. Formal Solution / Proof

Claim 2. *For $n \geq 1$,*

$$\Gamma_{\mathcal{H}_{2\text{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4}.$$

Proof. By Problem A1, realizable labelings are exactly the strings with no run of 1s, exactly one run of 1s, or exactly two runs of 1s.

Case 1: no runs of 1s. There is exactly one such labeling, namely $00\cdots0$. This contributes

$$1.$$

Case 2: exactly one run of 1s. Such a labeling is determined by choosing indices $a \leq b$ and declaring

$$y_i = \begin{cases} 1, & a \leq i \leq b, \\ 0, & \text{otherwise.} \end{cases}$$

So the number of one-run strings is the number of contiguous blocks in $\{1, \ldots, n\}$:

$$\sum_{a=1}^{n} \sum_{b=a}^{n} 1 = \frac{n(n+1)}{2} = \binom{n+1}{2}.$$

Case 3: exactly two runs of 1s. Such a string has the form

$$0\cdots0\,1\cdots1\,0\cdots0\,1\cdots1\,0\cdots0,$$

where the two 1-runs are nonempty and separated by at least one zero. A standard boundary encoding counts these strings by choosing four cut positions from the $n+1$ gaps before, between, and after the sample points. Concretely, choose

$$0 \leq s_1 < t_1 < s_2 < t_2 \leq n,$$

and place 1s on indices $s_1+1, \ldots, t_1$ and $s_2+1, \ldots, t_2$. This is a bijection between two-run labelings and 4-element subsets of the $n+1$ boundary positions. Therefore the number of two-run strings is

$$\binom{n+1}{4}.$$

Since the three cases are disjoint and exhaustive,

$$\Gamma_{\mathcal{H}_{2\text{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4}.$$

□

5. Conclusion

The exact growth function is

$$\Gamma_{\mathcal{H}_{2\text{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4}.$$

Its leading term is quartic, which already suggests the VC dimension will be finite and fairly small.

6. Lean Formalization

Lean: OPTIONAL.

If formalized, the target statement is:

$$\forall n, \quad \Gamma_{\mathcal{H}_{2\text{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4}.$$

7. Computational Check (if applicable)

The script `hw2/code/enumeration_check.py` enumerates all binary strings for $n \leq 6$, filters those with at most two runs of 1s, and checks that the counts agree with the formula above. The resulting values are:

n	1	2	3	4	5	6
$\Gamma_{\mathcal{H}_{2\text{int}}}(n)$	2	4	8	16	31	57

Problem A3

1. Restatement

Determine the VC dimension of $\mathcal{H}_{2\text{int}}$, prove that four points can be shattered, prove that five points cannot be shattered, and then compute the Halving mistake bound $\log_2 \Gamma_{\mathcal{H}_{2\text{int}}}(n)$. Finally compare that exact Halving bound with the Sauer–Shelah upper bound.

2. Explanation

The key concept is VC dimension. A class shatters a set when it realizes every binary labeling on that set. The run characterization from Problem A1 makes the shattering test especially transparent on ordered points.

3. Solution Plan

1. Show every labeling of four ordered points has at most two runs of 1s, so four points are shattered.

2. Exhibit a labeling of five ordered points with three runs of 1s, so five points are not shattered.
3. Conclude $\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) = 4$.
4. Substitute the exact growth function from A2 into the Halving bound and compare it to the Sauer–Shelah bound for VC dimension 4.

4. Formal Solution / Proof

Claim 3. $\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) = 4$.

Proof. Take any four ordered points $x_1 < x_2 < x_3 < x_4$. Every binary string of length four has at most two runs of 1s, because three runs would require at least five positions. By Problem A1, every one of the $2^4 = 16$ labelings is realizable by $\mathcal{H}_{2\mathrm{int}}$. Hence the set $\{x_1, x_2, x_3, x_4\}$ is shattered, so

$$\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) \geq 4.$$

Now consider any five ordered points $x_1 < \cdots < x_5$. The labeling

$$(1, 0, 1, 0, 1)$$

has three runs of 1s. By Problem A1, it is not realizable by a union of at most two intervals. Therefore no set of five ordered points is shattered, so

$$\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) \leq 4.$$

Combining the two inequalities gives

$$\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) = 4.$$

□

For the Halving algorithm on n ordered sample points, the exact number of candidate restrictions is $\Gamma_{\mathcal{H}_{2\mathrm{int}}}(n)$, so the exact Halving bound is

$$M_{\mathrm{Halving}} \leq \log_2 \Gamma_{\mathcal{H}_{2\mathrm{int}}}(n) = \log_2\left(1 + \binom{n+1}{2} + \binom{n+1}{4}\right).$$

Since $\mathrm{VCdim}(\mathcal{H}_{2\mathrm{int}}) = 4$, the Sauer–Shelah lemma gives the general upper bound

$$\Gamma_{\mathcal{H}_{2\mathrm{int}}}(n) \leq \sum_{k=0}^{4} \binom{n}{k}.$$

Therefore a coarser Halving bound is

$$M_{\mathrm{Halving}} \leq \log_2\left(\sum_{k=0}^{4} \binom{n}{k}\right).$$

5. Conclusion

The VC dimension is exactly 4. The exact Halving mistake bound is

$$\log_2\left(1 + \binom{n+1}{2} + \binom{n+1}{4}\right),$$

while Sauer–Shelah gives a more general but looser upper bound through $\sum_{k=0}^{4} \binom{n}{k}$.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

For $n = 4$, the computed growth value is

$$\Gamma_{\mathcal{H}_{2\text{int}}}(4) = 16 = 2^4,$$

which agrees with the shattering claim. For $n = 5$, the computed value is

$$\Gamma_{\mathcal{H}_{2\text{int}}}(5) = 31 < 32 = 2^5,$$

which agrees with the failure to shatter five points.

Problem A4

1. Restatement

Construct a hypothesis class whose VC dimension is exactly d and whose growth function on n points is exactly

$$\sum_{k=0}^{d} \binom{n}{k}.$$

Explain why this shows that the Sauer–Shelah bound can be tight.

2. Explanation

The key concept is a tight example for Sauer–Shelah. A natural class is the collection of subsets of the domain of size at most d. On an n-point sample, every labeling with at most d ones is realizable, and no labeling with more than d ones is.

3. Solution Plan

1. Define the class of all indicator functions of subsets of size at most d.
2. Prove that any d-point set is shattered.
3. Prove that no $(d+1)$-point set is shattered.
4. Count the realizable restrictions on an n-point sample.

4. Formal Solution / Proof

Let the domain X be any infinite set, and define

$$\mathcal{H}_{\leq d} = \{\mathbf{1}_S : S \subseteq X,\ |S| \leq d\}.$$

Claim 4. $\mathrm{VCdim}(\mathcal{H}_{\leq d}) = d$.

Proof. Take any d distinct points $x_1, \ldots, x_d$. For every labeling of these points, the positively labeled subset has size at most d, so it is itself a set S with $|S| \leq d$. Then $\mathbf{1}_S$ realizes that labeling. Hence every d-point set is shattered, so

$$\mathrm{VCdim}(\mathcal{H}_{\leq d}) \geq d.$$

Now take any $d+1$ distinct points $x_1, \ldots, x_{d+1}$. The all-ones labeling would require a set S containing all $d+1$ sample points, but no hypothesis in $\mathcal{H}_{\leq d}$ has support larger than d. So a $(d+1)$-point set cannot be shattered, and

$$\mathrm{VCdim}(\mathcal{H}_{\leq d}) \leq d.$$

Therefore

$$\mathrm{VCdim}(\mathcal{H}_{\leq d}) = d.$$

□

Now fix an n-point sample $x_1, \ldots, x_n$. A labeling is realizable by $\mathcal{H}_{\leq d}$ if and only if it has at most d ones. For each $k \in \{0, \ldots, d\}$, there are exactly $\binom{n}{k}$ such labelings with exactly k ones. Hence

$$\Gamma_{\mathcal{H}_{\leq d}}(n) = \sum_{k=0}^{d} \binom{n}{k}.$$

5. Conclusion

This class has VC dimension exactly d and attains the Sauer–Shelah upper bound exactly. Therefore the Sauer–Shelah inequality is, in general, tight.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

For small d and n, the expression $\sum_{k=0}^{d} \binom{n}{k}$ can be checked directly by counting the binary strings of length n with at most d ones. No separate repository artifact is required here because the main enumeration script already illustrates the same type of counting argument.

Problem A5

1. Restatement

Audit an AI-generated proof about the class $\mathcal{H}_{2\mathrm{int}}$. Identify the flaw in its counting argument, explain why the proof is invalid, and replace it with a correct statement based on the two-run characterization.

2. Explanation

The key concept is structural counting. A bad AI proof often counts possible "switch points" as though label changes could occur arbitrarily, but unions of two intervals impose strong interval constraints. The proof must respect the geometry of the class.

3. Solution Plan

1. State the flaw: arbitrary switch-point counting ignores interval structure.
2. Explain why this leads to overcounting.
3. Replace the invalid claim with the correct run-based characterization.
4. Deduce the correct growth-function statement from Problem A2.

4. Formal Solution / Proof

Suppose the AI argument says that a union of two intervals can be counted by choosing up to four switch points independently among the $n-1$ gaps between sample points, and therefore claims a count on the order of

$$\sum_{k=0}^{4} \binom{n-1}{k}.$$

This argument is flawed for two reasons.

First, it treats label changes as arbitrary, but a union of two intervals does not produce an arbitrary set of switch points. The positive region on a sorted sample must be the union of at most two contiguous blocks, so the valid patterns are exactly those with at most two runs of 1s.

Second, even when a pattern has at most four changes, the locations are not independent choices. The interval structure forces them to occur in a very specific alternating order, and the relevant combinatorial object is not "all subsets of at most four switch points" but rather "boundary encodings of at most two runs of 1s".

The correct statement is:

> For n ordered sample points, the realizable labelings of $\mathcal{H}_{2\text{int}}$ are exactly the binary strings with at most two runs of 1s. Consequently,
>
> $$\Gamma_{\mathcal{H}_{2\text{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4}.$$

This follows immediately from Problem A1 for the structural characterization and from Problem A2 for the exact count.

5. Conclusion

The AI proof is invalid because it counts unconstrained switch patterns instead of interval-consistent runs. The correct reasoning is run-based, not arbitrary change-point counting.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

The enumeration script provides a concrete reality check: for $n \le 6$, the brute-force counts agree with the two-run formula and do not agree with the much larger naive switch-point count.

Part B

Throughout Part B, define the quadratic-threshold class

$$\mathcal{H}_{\text{quad}} = \left\{ h_{a,b,c} : x \mapsto \mathbf{1}\left[ax^2 + bx + c \ge 0\right] : (a,b,c) \in \mathbb{R}^3 \setminus \{(0,0,0)\} \right\}.$$

We again study restrictions to ordered samples $x_1 < \cdots < x_n$.

Problem B1

1. Restatement

Prove the general upper bound: if a hypothesis class can be represented as homogeneous linear threshold functions in $\mathbb{R}^D$, then its VC dimension is at most D.

2. Explanation

The key concept is linear dependence. A set of $D+1$ vectors in $\mathbb{R}^D$ is always linearly dependent. That dependence can be converted into a labeling that no homogeneous linear threshold can realize, giving the contradiction needed for the VC-dimension upper bound.

3. Solution Plan

1. Assume $D+1$ points are shattered after applying the representation map ϕ.
2. Use linear dependence among their feature vectors.
3. Turn that dependence relation into a forbidden labeling.
4. Conclude that shattering $D+1$ points is impossible.

4. Formal Solution / Proof

Theorem 5. *Let X be any domain and let $\phi : X \to \mathbb{R}^D$. Define*

$$\mathcal{H}_\phi = \left\{ x \mapsto \mathbf{1}[\langle w, \phi(x) \rangle \geq 0] : w \in \mathbb{R}^D \right\}.$$

Then

$$\mathrm{VCdim}(\mathcal{H}_\phi) \leq D.$$

Proof. Assume for contradiction that $\mathcal{H}_\phi$ shatters $D+1$ points $x_1, \ldots, x_{D+1} \in X$. Write

$$v_i = \phi(x_i) \in \mathbb{R}^D.$$

Since $D+1$ vectors in $\mathbb{R}^D$ are linearly dependent, there exists a nontrivial relation

$$\alpha_1 v_1 + \cdots + \alpha_{D+1} v_{D+1} = 0$$

with coefficients not all zero. Remove any zero coefficients if necessary and replace the relation by a minimal one, so every remaining coefficient is nonzero.

Define the target labeling on the remaining indexed points by

$$y_i = \begin{cases} 1, & \alpha_i > 0, \\ 0, & \alpha_i < 0. \end{cases}$$

Because the original set is shattered, this labeling must be realizable by some weight vector $w \in \mathbb{R}^D$. Since the sample is finite, we may perturb w slightly if necessary so that the realizing inequalities are strict on the sample:

$$\langle w, v_i \rangle > 0 \quad \text{when } \alpha_i > 0, \qquad \langle w, v_i \rangle < 0 \quad \text{when } \alpha_i < 0.$$

Now take the dot product of w with the dependence relation:

$$0 = \left\langle w, \sum_{i=1}^{D+1} \alpha_i v_i \right\rangle = \sum_{i=1}^{D+1} \alpha_i \langle w, v_i \rangle.$$

Every term in the final sum is strictly positive, because α_i and $\langle w, v_i \rangle$ have the same sign. Hence the sum is strictly positive, a contradiction.

Therefore no set of $D+1$ points can be shattered, so

$$\mathrm{VCdim}(\mathcal{H}_\phi) \leq D.$$

□

5. Conclusion

Any class represented by homogeneous linear threshold functions in $\mathbb{R}^D$ has VC dimension at most D. This is the general fact needed in Problem B2.

6. Lean Formalization

Lean: YES.

Target theorem to formalize:

$$\text{If } \mathcal{H}_\phi = \left\{ x \mapsto \mathbf{1}[\langle w, \phi(x) \rangle \geq 0] : w \in \mathbb{R}^D \right\}, \text{ then } \mathrm{VCdim}(\mathcal{H}_\phi) \leq D.$$

Repository artifact: `hw2/lean/vc_dimension_bound.lean`.

7. Computational Check (if applicable)

No numerical check is needed here because the argument is purely linear algebraic.

Problem B2

1. Restatement

Show that quadratic threshold functions on the real line admit a linear representation in $\mathbb{R}^3$ under the map

$$\phi(x) = (x^2, x, 1),$$

and conclude from Problem B1 that their VC dimension is at most 3.

2. Explanation

The key concept is feature lifting. The quadratic polynomial $ax^2 + bx + c$ becomes a dot product with the lifted feature vector $(x^2, x, 1)$. Once that representation is established, Problem B1 applies immediately.

3. Solution Plan

1. Define the feature map $\phi(x) = (x^2, x, 1)$.
2. Rewrite every quadratic polynomial as a dot product in $\mathbb{R}^3$.
3. Identify the quadratic-threshold class with a linear-threshold class in $\mathbb{R}^3$.
4. Invoke Problem B1 to obtain the VC-dimension upper bound.

4. Formal Solution / Proof

Define

$$\phi : \mathbb{R} \to \mathbb{R}^3, \qquad \phi(x) = (x^2, x, 1).$$

For any $(a, b, c) \in \mathbb{R}^3$, let

$$w = (a, b, c) \in \mathbb{R}^3.$$

Then

$$\langle w, \phi(x) \rangle = ax^2 + bx + c.$$

Therefore

$$h_{a,b,c}(x) = \mathbf{1}\left[ax^2 + bx + c \geq 0\right] = \mathbf{1}[\langle w, \phi(x) \rangle \geq 0]\,.$$

So $\mathcal{H}_{\text{quad}}$ is a linear-threshold class in $\mathbb{R}^3$. By Problem B1,

$$\text{VCdim}(\mathcal{H}_{\text{quad}}) \leq 3.$$

5. Conclusion

Quadratic thresholds on $\mathbb{R}$ lift to homogeneous linear thresholds in $\mathbb{R}^3$, and therefore

$$\text{VCdim}(\mathcal{H}_{\text{quad}}) \leq 3.$$

6. Lean Formalization

Lean: YES.

Target theorem to formalize:

$$\forall a, b, c, x \in \mathbb{R}, \qquad ax^2 + bx + c = \langle (a, b, c), (x^2, x, 1) \rangle.$$

Together with Problem B1, this yields $\text{VCdim}(\mathcal{H}_{\text{quad}}) \leq 3$.

Repository artifact: `hw2/lean/quadratic_embedding.lean`.

7. Computational Check (if applicable)

For small samples, the script in `hw2/code/enumeration_check.py` computes the restriction counts predicted later in Problem B4. This does not prove the embedding, but it provides a sanity check on the induced pattern count.

Problem B3

1. Restatement

Describe exactly which binary labelings of an ordered sample $x_1 < \cdots < x_n$ can arise from quadratic thresholds. The required argument must use the fact that a quadratic has at most two real roots and must describe all valid patterns carefully.

2. Explanation

The key concept is root structure. A quadratic polynomial can only change sign when crossing a real root, and it has at most two real roots. Consequently the set on which the quadratic is nonnegative is either an interval, the complement of an interval, all of $\mathbb{R}$, or the empty set. On an ordered sample this translates into a precise family of binary strings.

3. Solution Plan

1. Use the root description of quadratic polynomials.
2. Translate each geometric possibility into a binary pattern on a sorted sample.
3. Show that the valid 1-sets are exactly: empty, one contiguous run, or a prefix together with a suffix.
4. Verify the converse by constructing an appropriate quadratic in each case.

4. Formal Solution / Proof

Theorem 6. *For an ordered sample $x_1 < \cdots < x_n$, a binary labeling $(y_1, \ldots, y_n) \in \{0,1\}^n$ is realizable by $\mathcal{H}_{\text{quad}}$ if and only if the set of indices with label 1 is one of the following:*

1. *the empty set,*
2. *a single contiguous block,*
3. *the union of a prefix and a suffix separated by a nonempty middle block of zeros.*

Proof. Let $q(x) = ax^2 + bx + c$.

If $a = 0$, then q is linear or constant, so the set $\{x : q(x) \geq 0\}$ is either empty, all of $\mathbb{R}$, or a half-line. On an ordered sample this produces either the empty labeling, the all-ones labeling, a prefix of ones, or a suffix of ones. These are already covered by the three stated cases.

Now assume $a \neq 0$. A quadratic has at most two real roots.

If q has no real roots, then its sign never changes, so the labeling is either all ones or all zeros.

If q has two real roots $r_1 \leq r_2$, then:

- when $a < 0$, the set $\{x : q(x) \geq 0\}$ is the interval $[r_1, r_2]$, so the 1-labels form one contiguous block on the sample;
- when $a > 0$, the set $\{x : q(x) \geq 0\}$ is the exterior region $(-\infty, r_1] \cup [r_2, \infty)$, so the 1-labels form a prefix together with a suffix.

This proves that every realizable labeling has one of the three stated forms.

For the converse, we explicitly realize each form.

The empty labeling is realized by $q(x) = -1$, and the all-ones labeling by $q(x) = 1$.

If the 1-labels form a single contiguous block from x_a through x_b, choose roots $r_1 < x_a$ and $r_2 > x_b$ lying between neighboring sample points so that

$$q(x) = -(x - r_1)(x - r_2)$$

is nonnegative exactly on that block.

If the 1-labels form a prefix together with a suffix, with a middle zero block from x_a through x_b, choose roots $r_1 < x_a$ and $r_2 > x_b$ again between neighboring sample points, and define

$$q(x) = (x - r_1)(x - r_2).$$

Then $q(x) \geq 0$ exactly outside the interval (r_1, r_2), so it realizes that prefix–suffix pattern.

Thus the listed patterns are exactly the realizable restrictions of $\mathcal{H}_{\text{quad}}$. □

5. Conclusion

Quadratic thresholds do not produce arbitrary sign-change patterns. Their restrictions are controlled by the root structure of quadratics: the 1-set is either empty, one block, or a prefix together with a suffix.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

The enumeration script verifies for $n \leq 6$ that the count of strings in these three families matches the closed form derived in Problem B4.

Problem B4

1. Restatement

Use the structural characterization from Problem B3 to compute the growth function of $\mathcal{H}_{\text{quad}}$ on n ordered points, determine its VC dimension, and compare the exact growth count with the Sauer–Shelah bound.

2. Explanation

The key concept is again careful case counting. From B3, the valid labelings are empty, one run, or a prefix together with a suffix. Counting those patterns gives the exact growth function and also reveals the exact VC dimension.

3. Solution Plan

1. Count the empty labeling.
2. Count one-run labelings by choosing a contiguous block.
3. Count prefix–suffix labelings by choosing the interior zero block.
4. Sum the disjoint cases and simplify.
5. Check shattering at $n = 3$ and failure at $n = 4$.

4. Formal Solution / Proof

By Problem B3, the realizable labelings fall into three disjoint classes.

Case 1: empty labeling. There is exactly one such labeling.

Case 2: one contiguous run of 1s. The number of contiguous blocks in n ordered positions is

$$\binom{n+1}{2}.$$

This count includes the all-ones labeling.

Case 3: prefix together with suffix. Such a labeling is determined by a nonempty contiguous middle block of zeros that is strictly interior to the sample. If that zero block runs from index a to index b, then

$$2 \le a \le b \le n-1.$$

The number of such interior zero blocks is the number of contiguous blocks in $\{2, \dots, n-1\}$, namely

$$\binom{n-1}{2}.$$

Adding the cases gives

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) = 1 + \binom{n+1}{2} + \binom{n-1}{2}.$$

Since

$$\binom{n+1}{2} + \binom{n-1}{2} = \frac{n(n+1)}{2} + \frac{(n-1)(n-2)}{2} = n^2 - n + 1,$$

we obtain the closed form

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) = n^2 - n + 2.$$

To compute the VC dimension, observe first that

$$\Gamma_{\mathcal{H}_{\text{quad}}}(3) = 3^2 - 3 + 2 = 8 = 2^3.$$

So three ordered points are shattered. On the other hand,

$$\Gamma_{\mathcal{H}_{\text{quad}}}(4) = 4^2 - 4 + 2 = 14 < 16 = 2^4,$$

so four points are not shattered. Hence

$$\text{VCdim}(\mathcal{H}_{\text{quad}}) = 3.$$

This matches the upper bound from Problem B2. Since $\text{VCdim}(\mathcal{H}_{\text{quad}}) = 3$, Sauer–Shelah gives

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) \le \sum_{k=0}^{3} \binom{n}{k}.$$

The exact count $n^2 - n + 2$ is sharper for this specific class.

5. Conclusion

The exact growth function is

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) = n^2 - n + 2,$$

and the VC dimension is exactly 3. The exact count is smaller than the general Sauer–Shelah upper bound, as expected.

6. Lean Formalization

Lean: OPTIONAL.

If formalized, the target statements are

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) = n^2 - n + 2 \qquad \text{and} \qquad \text{VCdim}(\mathcal{H}_{\text{quad}}) = 3.$$

7. Computational Check (if applicable)

The script `hw2/code/enumeration_check.py` checks the pattern count for $n \leq 6$. The resulting values are:

n	1	2	3	4	5	6
$\Gamma_{\mathcal{H}_{\text{quad}}}(n)$	2	4	8	14	22	32

Problem B5

1. Restatement

Audit an AI proof about quadratic thresholds. Identify the flaw in an argument that counts arbitrary change points, explain why that argument is invalid, and replace it with correct reasoning based on the root structure of quadratics.

2. Explanation

The key concept is that not every pattern with a small number of sign changes is produced by a quadratic. The shape of a quadratic forces the positive region to be either an interval or the complement of an interval. Any proof that ignores this curvature constraint overcounts the hypothesis class.

3. Solution Plan

1. Identify the flaw: arbitrary change-point counting ignores the shape of quadratic level sets.
2. Explain the missing constraint: a quadratic has at most two roots and a fixed opening direction.
3. Replace the argument with the interval / complement-of-interval classification from Problem B3.
4. Deduce the correct growth function from Problem B4.

4. Formal Solution / Proof

Suppose an AI argument claims that because a quadratic has at most two roots, we can choose up to two sign-change locations arbitrarily among the $n-1$ sample gaps, and therefore count all strings with at most two changes. This is incorrect.

The flaw is that "at most two changes" is necessary but not sufficient for a quadratic threshold. The polynomial shape matters. For example, a pattern like

$$0, 1, 1, 0$$

has two changes and is realizable, but a pattern like

$$1, 0, 0, 1$$

is also realizable for a completely different geometric reason: it comes from a quadratic that opens upward and is nonnegative outside an interval. Counting "change points" alone misses this distinction and does not encode whether the quadratic is selecting an interval or its complement.

The correct statement is the one proved in Problem B3: on an ordered sample, the realisable 1-sets are exactly the empty set, one contiguous block, or a prefix together with a suffix. Consequently, by Problem B4,

$$\Gamma_{\mathcal{H}_{\text{quad}}}(n) = n^2 - n + 2.$$

5. Conclusion

The invalid AI proof overcounts by ignoring the interval structure imposed by quadratic roots. The right argument is root-based and shape-based, not arbitrary change-point counting.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

The small-n enumeration confirms the exact formula $n^2 - n + 2$ and serves as a quick check against incorrect overcounting heuristics.

Part C

Problem C1

1. Restatement

Write an AI usage report addressing where AI was used, which suggestions were accepted, which suggestions were rejected or modified, how correctness was verified, and how the workflow improved over the course of the assignment.

2. Explanation

This part is reflective rather than mathematical. The goal is to document how AI contributed to the assignment while still making clear that the final arguments, counts, and conclusions were checked carefully.

3. Solution Plan

1. Describe where AI entered the workflow.
2. Give one accepted suggestion and why it was useful.
3. Give one rejected or revised suggestion and why it needed correction.
4. Explain the verification process.
5. Summarize workflow improvements.

4. Formal Solution / Proof

I used AI in several parts of this assignment. For the mathematical write-up, I used it to help structure proofs, organize the document, and identify cleaner ways to present the combinatorial cases. For the computational support, I used it to help design a small enumeration script that checks the growth-function formulas on small values of n. I also used AI to identify which parts of the assignment were the best targets for Lean formalization.

One suggestion I accepted was to organize each problem into the sequence "restatement, explanation, plan, proof, conclusion." I kept this structure because it made the final document much easier to audit: the intuition is separated from the rigorous proof, and each formula appears only after the relevant case split has been explained.

One suggestion I rejected or modified concerned proof style. An earlier draft counted restrictions by talking loosely about possible "switch points" in the binary labeling. I revised that approach because it risked overcounting. For the two-interval class, the correct invariant is the number of runs of 1s, and for the quadratic class the correct invariant comes from roots and the shape of the nonnegative region. Replacing the switch-point language with the more structured run-based and root-based arguments made the counting precise.

I verified correctness in several ways. First, I checked each combinatorial formula against the structural characterization proved immediately before it. Second, I used small-n computational enumeration to confirm the formulas

$$\Gamma_{\mathcal{H}_{2\mathrm{int}}}(n) = 1 + \binom{n+1}{2} + \binom{n+1}{4} \qquad \text{and} \qquad \Gamma_{\mathcal{H}_{\mathrm{quad}}}(n) = n^2 - n + 2$$

for $n \leq 6$. Third, I cross-checked the VC-dimension conclusions against the exact values of the growth functions at the critical sample sizes. Finally, I made sure the Lean targets matched the mathematically central statements rather than peripheral bookkeeping lemmas.

The workflow improved as the assignment progressed. Early on, AI was most useful for brainstorming proof organization and computational checks. Later, I used it more selectively: mainly for tightening exposition, flagging potential overcounting, and isolating the statements that were worth formalizing in Lean. This made the overall process more disciplined and reduced the amount of time spent rewriting informal arguments after the fact.

5. Conclusion

AI was useful for organization, drafting, and small verification support, but the final correctness relied on explicit proof checking, small-case computation, and careful revision of any overly loose or overconfident suggestions.

6. Lean Formalization

Lean: NO.

7. Computational Check (if applicable)

The computational checks mentioned in this report are implemented in `hw2/code/enumeration_check.py`, with outputs stored under `hw2/figures/`.

Repository Artifacts and Mapping

Supporting Repository Artifacts

The supporting files for this assignment are:

- `hw2/writeup.tex`: the source of this write-up.
- `hw2/writeup.pdf`: the compiled submission PDF.
- `hw2/lean/vc_dimension_bound.lean`: Lean target for Problem B1.
- `hw2/lean/quadratic_embedding.lean`: Lean target for Problem B2.
- `hw2/code/enumeration_check.py`: small-case verification script for Problems A2 and B4.
- `hw2/figures/small_n_growth.csv`: generated table of small-n growth values.
- `hw2/figures/small_n_growth.png`: generated comparison plot.

Write-Up to Repository Mapping

Write-up section	Supporting artifact
Problem A2	`hw2/code/enumeration_check.py` and `hw2/figures/small_n_growth.csv`
Problem A3	`hw2/code/enumeration_check.py` for the values $\Gamma_{\mathcal{H}_{2\text{int}}}(4) = 16$ and $\Gamma_{\mathcal{H}_{2\text{int}}}(5) = 31$
Problem B1	`hw2/lean/vc_dimension_bound.lean`
Problem B2	`hw2/lean/quadratic_embedding.lean`
Problem B4	`hw2/code/enumeration_check.py`, `hw2/figures/small_n_growth.csv`, and `hw2/figures/small_n_growth.png`
All parts	`hw2/writeup.tex` and `hw2/writeup.pdf`

Lean Targets Summary

The parts that should be formalized in Lean are:

- Problem B1: the general theorem that a class represented by homogeneous linear thresholds in $\mathbb{R}^D$ has VC dimension at most D.
- Problem B2: the quadratic embedding

$$ax^2 + bx + c = \langle (a, b, c), (x^2, x, 1) \rangle.$$

The combinatorial counting arguments in Parts A and B are good mathematical targets in principle, but they are not the most efficient first formalizations for this repository. The linear-representation results are cleaner and more reusable.